

Strong and Compact Policies for Submodular Markov Decision Processes

via LP-Based Submodular Orienteering

Lars Rohwedder

University of Southern Denmark

Rico Zenklusen

ETH Zurich

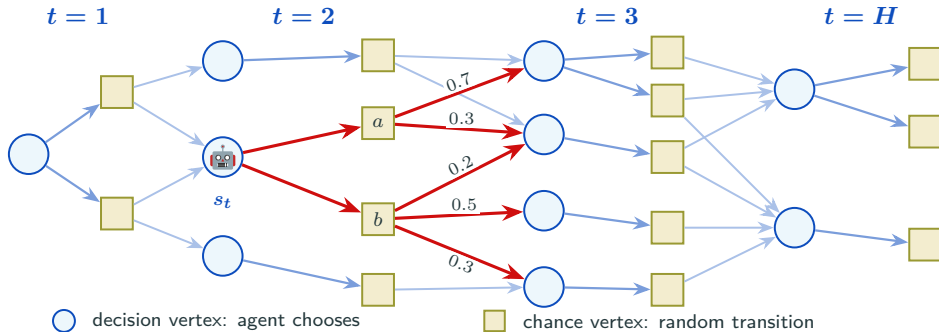


Funded by
the European Union



European Research Council
Established by the European Commission

Markov Decision Processes

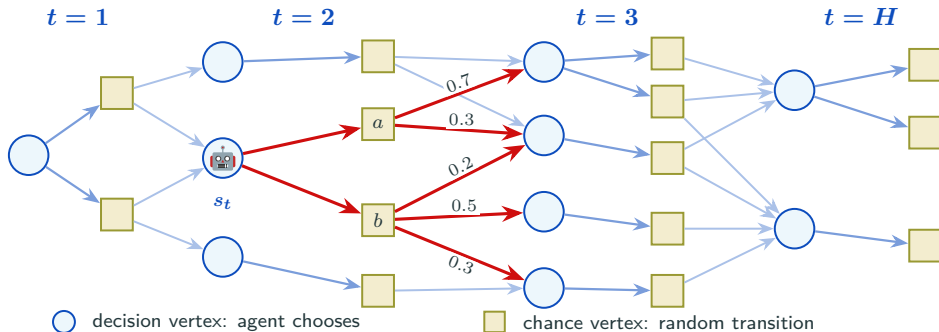


Traditional additive rewards

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=1}^H r_t(s_t, a_t) \right]$$

An optimal policy follows from **Bellman's recursion and dynamic programming**.

Submodular Markov Decision Processes



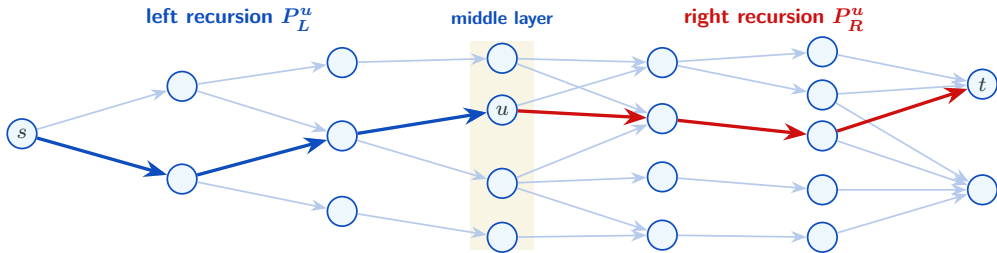
Submodular reward over the whole trajectory

$$\max_{\pi} \mathbb{E}_{\pi} \left[f \left(\{s_1, a_1, \dots, s_H, a_H\} \right) \right]$$
$$f : 2^{S \cup A} \rightarrow \mathbb{R}_{\geq 0} \text{ monotone submodular}$$

Wang, Zhang, Chaplot, Garagić, and Salakhutdinov (arXiv 2020). *Planning with submodular objective functions*.
Prajapat, Mutny, Zeilinger, and Krause (ICLR 2024). *Submodular reinforcement learning*.

Recursive Greedy (Chekuri-Pál, 2005)

Submodular orienteering: maximize $f(P)$ over s - t paths P , with $f \geq 0$ monotone submodular.



1. Try each vertex u in the middle layer.
2. **Left:** recursively find an s - u path P_L^u using f .
3. **Right:** recursively find a u - t path P_R^u using the marginal reward $f(\cdot \mid P_L^u)$.
4. Return the best concatenation over all u . Solve a single edge directly.

Guarantee: $O(\log n)$ approximation in $n^{O(\log n)}$ time, where $n = |V|$.

Main contributions

LP-based submodular orienteering (n vertices)

Approximation	Computation time
$O(\log n)$	$n^{O(\log n)}$
$O(n^\varepsilon)$	$n^{O(1/\varepsilon)}$

Reward guarantees hold in expectation.

Main contributions

LP-based submodular orienteering (n vertices)

Approximation	Computation time
$O(\log n)$	$n^{O(\log n)}$
$O(n^\varepsilon)$	$n^{O(1/\varepsilon)}$

Reward guarantees hold in expectation.

Submodular MDPs, via the orienteering LP

Previous best approximation: $O(H)$.

Approximation	Computation time	Past vertices per decision
$O(\log H)$	$n^{O(\log H)}$	$O(\log H)$
$O(H^\varepsilon)$	$n^{O(1/\varepsilon)}$	$O(1/\varepsilon)$

Here $n = \text{poly}(|S|, |A|, H)$.

The LP and recursive rounding

A Sherali–Adams type LP

Graph $G = (V, E)$ has layers $V_1, \dots, V_H$, with edges from V_i to V_{i+1} .

Base LP $Q^{(0)}$

Variables x_v , for $v \in V$.

$$\sum_{v \in V_i} x_v = 1 \quad \text{for each layer } i,$$

$$x_u + x_v \leq 1 \quad \text{if } (u, v) \in (V_i \times V_{i+1}) \setminus E,$$

$$0 \leq x_v \leq 1 \quad \text{for every } v \in V.$$

A Sherali–Adams type LP

Graph $G = (V, E)$ has layers $V_1, \dots, V_H$, with edges from V_i to V_{i+1} .

Sherali–Adams lift $Q^{(r)}$

Fix $r \geq 1$. Joint variables x_I for $I \subseteq V$, $|I| \leq r+1$. For every I with $|I| \leq r$:

$$\sum_{v \in V_i} x_{I \cup \{v\}} = x_I \quad \text{for each layer } i,$$

$$x_{I \cup \{u\}} + x_{I \cup \{v\}} \leq x_I \quad \text{if } (u, v) \in (V_i \times V_{i+1}) \setminus E,$$

$$0 \leq x_{I \cup \{v\}} \leq x_I \quad \text{for every } v \in V, \quad x_\emptyset = 1.$$

Lifting: multiply constraints by $\prod_{w \in I} x_w$ and linearize. Intended meaning of x_I : $\Pr[I \subseteq P]$.

A Sherali–Adams type LP

Graph $G = (V, E)$ has layers $V_1, \dots, V_H$, with edges from V_i to V_{i+1} .

Sherali–Adams lift $Q^{(r)}$

Fix $r \geq 1$. Joint variables x_I for $I \subseteq V$, $|I| \leq r+1$. For every I with $|I| \leq r$:

$$\sum_{v \in V_i} x_{I \cup \{v\}} = x_I \quad \text{for each layer } i,$$

$$x_{I \cup \{u\}} + x_{I \cup \{v\}} \leq x_I \quad \text{if } (u, v) \in (V_i \times V_{i+1}) \setminus E,$$

$$0 \leq x_{I \cup \{v\}} \leq x_I \quad \text{for every } v \in V, \quad x_\emptyset = 1.$$

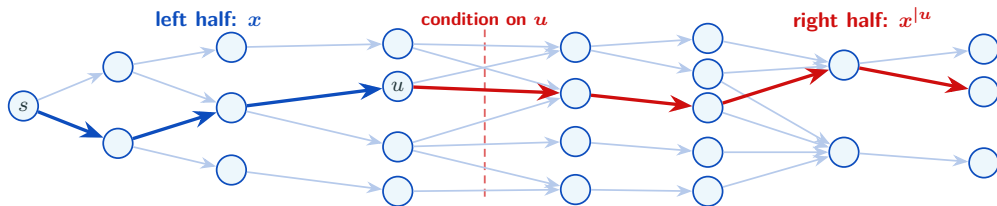
Lifting: multiply constraints by $\prod_{w \in I} x_w$ and linearize. Intended meaning of x_I : $\Pr[I \subseteq P]$.

Conditioning: $x \in Q^{(r)}$ and $x_u > 0$

$$(x^{|u})_I = \frac{x_{I \cup \{u\}}}{x_u} \implies x^{|u} \in Q^{(r-1)}, \quad (x^{|u})_u = 1.$$

Recursive randomized rounding

Input: $x \in Q^{(r)}$ on $H = 2^r$ layers, for an integer $r \geq 0$. If $H > 1$, split the layers into equal halves L_L, L_R .



Base case: if $H = 1$, sample and return vertex v with probability x_v .

1. **Left half:** recursively round x restricted to L_L , with one fewer LP level, to obtain P_L .
2. Let u be the last vertex of P_L . **Right half:** recursively round $x|u$ restricted to L_R to obtain P_R .
3. Return $P_L \circ P_R$.

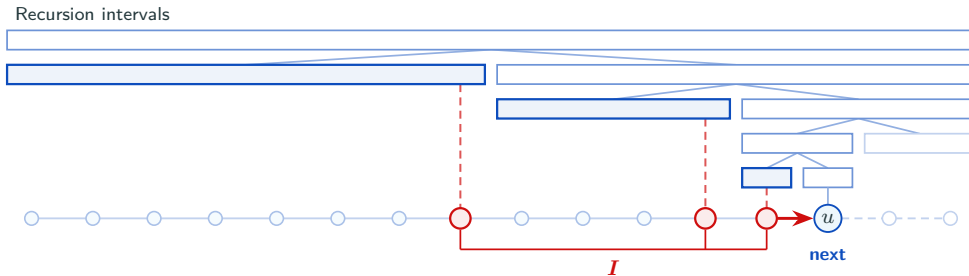
Properties

A layer-spanning path with $\Pr[v \in P] = x_v$ for all v . Conditioning uses only past vertices.

The policy

Implementing RRR online

Given an LP solution x , **sample vertices in order from left to right.**



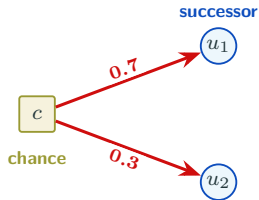
Next step: condition on I

$$\Pr[\text{next vertex} = u \mid \text{observed trajectory}] = (x^I)_u = \frac{x_{I \cup \{u\}}}{x_I}.$$

Exactly the same path distribution as RRR.

Adding chance vertices to the LP

At decision vertices, the policy chooses the next vertex. At a **chance vertex** $c \in C$, the environment uses the given distribution p_c .



Add to $Q^{(r)}$

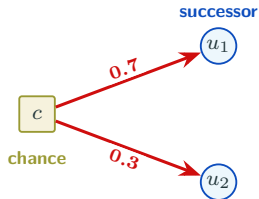
$$x_{\{c,u\}} = x_c p_c(u)$$

For every $c \in V_i \cap C$, $u \in V_{i+1}$.

Every valid policy's trajectory moments satisfy these constraints, including those of the optimum.

Adding chance vertices to the LP

At decision vertices, the policy chooses the next vertex. At a **chance vertex** $c \in C$, the environment uses the given distribution p_c .



Add to $Q^{(r)}$

$$x_{I \cup \{u\}} = x_I p_c(u)$$

For every $c \in V_i \cap C$, $u \in V_{i+1}$, and

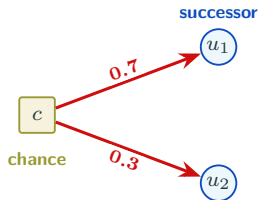
$$\{c\} \subseteq I \subseteq V_1 \cup \dots \cup V_i, \quad |I| \leq r.$$

The conditioning set contains c and only past or current vertices.

Every valid policy's trajectory moments satisfy these constraints, including those of the optimum.

Adding chance vertices to the LP

At decision vertices, the policy chooses the next vertex. At a **chance vertex** $c \in C$, the environment uses the given distribution p_c .



Add to $Q^{(r)}$

$$x_{I \cup \{u\}} = x_I p_c(u)$$

For every $c \in V_i \cap C$, $u \in V_{i+1}$, and

$$\{c\} \subseteq I \subseteq V_1 \cup \dots \cup V_i, \quad |I| \leq r.$$

The conditioning set contains c and only past or current vertices.

Every valid policy's trajectory moments satisfy these constraints, including those of the optimum.

The online rule matches the environment

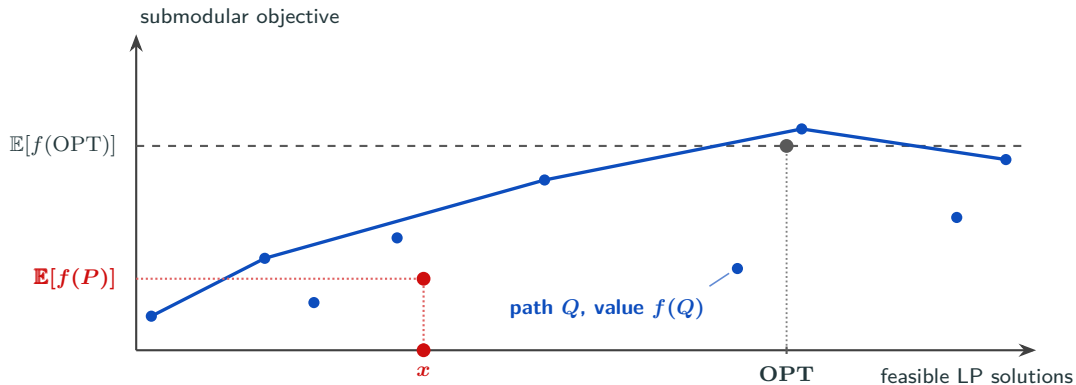
If $v_\ell = c \in C$, then $c \in I$, so

$$\Pr[v_{\ell+1} = u \mid v_1, \dots, v_\ell] = \frac{x_{I \cup \{u\}}}{x_I} = p_c(u).$$

Round or Cut

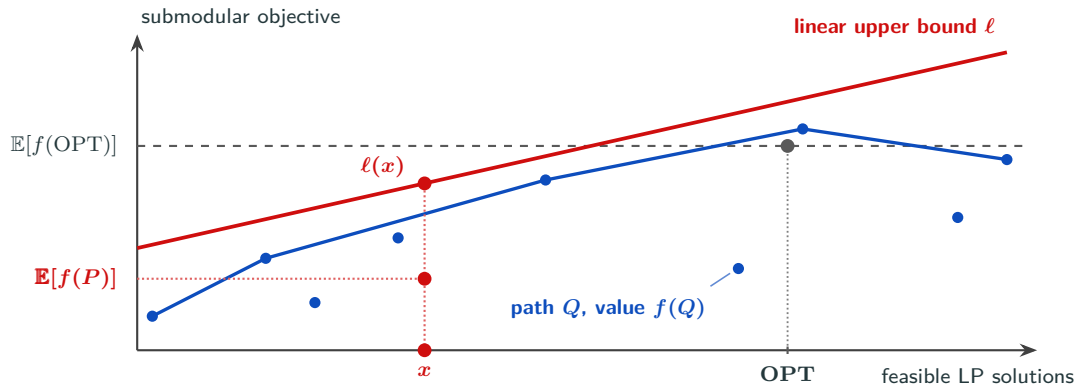
Round or Cut: the idea

Sample a path $P \sim \text{RRR}(x)$ from the current LP solution x .



Round or Cut: the idea

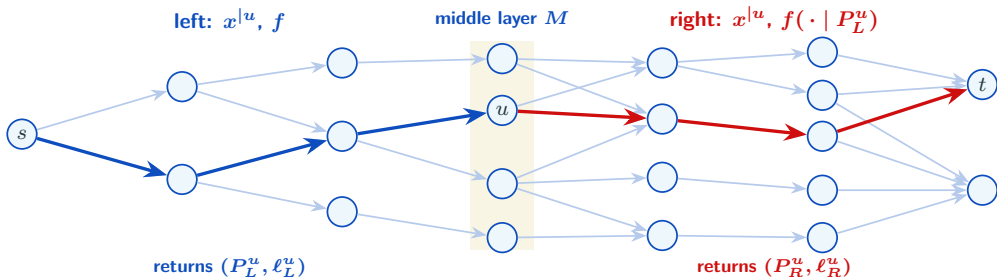
Along with $P \sim \text{RRR}(x)$, we output a **linear upper bound** ℓ on the reward of every path.



$$\mathbb{E}_{P \sim \text{RRR}(x)}[f(P)] \geq \frac{\mathbb{E}[\ell(x)]}{r+1}.$$

Constructing ℓ : a simplified case

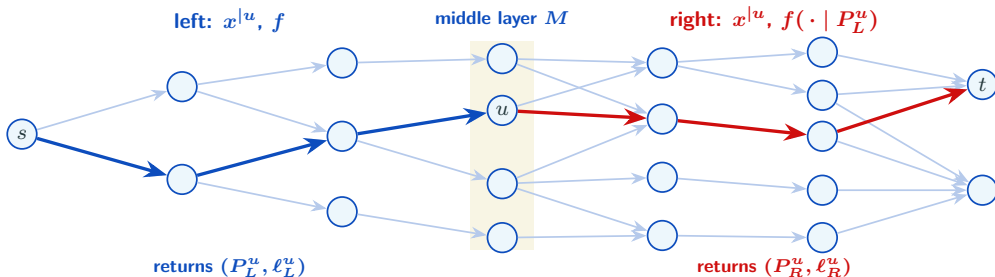
We switch to a **different, simplified offline setting** to illustrate the main ideas.



1. For each middle vertex u , **condition on u** and recurse on the **left** to obtain P_L^u .
2. Recurse on the right with the **marginal reward** $f(\cdot | P_L^u)$ to obtain P_R^u .
3. Sample u with probability x_u and join the two paths at u .

Constructing ℓ : a simplified case

We switch to a **different, simplified offline setting** to illustrate the main ideas.



1. For each middle vertex u , **condition on u** and recurse on the **left** to obtain P_L^u .
2. Recurse on the right with the **marginal reward** $f(\cdot | P_L^u)$ to obtain P_R^u .
3. Sample u with probability x_u and join the two paths at u .

$$\ell(x) = \sum_{u \in M} x_u \left[\ell_L^u(x^{|u}) + \mathbf{f}(P_L^u) + \ell_R^u(x^{|u}) \right].$$

Summary

Let $H = 2^r$, so $r = \log_2 H$. Fix $0 < \varepsilon < 1$.

1. Compute x by Round or Cut

In $n^{O(r)}$ time, compute $x \in Q^{(r)}$ satisfying the chance constraints. With high probability, the returned x has

$$\mathbf{E}_{P \sim \text{RRR}(x)}[f(P)] \geq \frac{1 - \varepsilon}{r + 1} \mathbf{E}[f(\text{OPT})].$$

2. Use this same x to implement RRR online

Use conditional LP probabilities at decision vertices.

The trajectory follows $\text{RRR}(x)$ and inherits the expected-reward guarantee above.

Idea for polynomial time variant: split into more blocks per recursion level. Fewer levels, $r = O(1/\varepsilon)$, give a smaller LP:

$$n^{O(1/\varepsilon)} \text{ time} \quad \text{and} \quad O(H^\varepsilon) \text{ approximation} \quad \text{for fixed } \varepsilon > 0.$$